

Capital.com Async Trading Platform – Documentation

Generated by Claude Code on 2026-05-09

Capital.com Async Trading Platform – Documentation

Project Overview

This repository implements a production-oriented, **async-first** trading platform for Capital.com. The architecture follows the layered, event-driven design you described:

| Layer | Responsibility | |-----|-----| | **broker** | Direct HTTP/WebSocket communication with Capital.com – authentication, session handling, order & position endpoints. | | **market** | Ingests raw tick data, builds candles, detects signals, emits typed events. | | **risk** | Central gatekeeper that validates every order against position size, daily loss caps, stale tick checks, etc. | | **services** | Orchestrates workflows (order submission, reconciliation, startup/shutdown, health checks). | | **db** | SQLAlchemy async ORM models and repository abstractions for persistence. | | **core** | Configuration (pydantic settings), structured logging (structlog), global trading state, base event contract. | | **api** | FastAPI HTTP endpoints exposing health, position query, etc. | | **queue_manager** | Simple in-memory queues (price_queue, order_queue, alert_queue). |

The code is fully async, uses **Python 3.11+**, and is container-ready (Dockerfile & compose placeholder included).

Key Files & Their Roles

- `app/core/config.py` – typed configuration loaded from `.env`. - `app/core/logging.py` – configures JSON-structured logs via `structlog`. - `app/core/events.py` – base `BaseEvent` dataclass for all events. - `app/core/state.py` – runtime `TradingState` and `TradingMode` enum. - `app/db/models/\*.py` – ORM models (`Position`, `Order`, ...). - `app/db/repositories/\*.py` – repository layer (no raw SQL in services). - `app/broker/client.py` – generic async HTTP client with `backoff` retry. - `app/broker/session.py` – token lifecycle (refreshes 5 min before expiry). - `app/broker/websocket.py` – price WebSocket; pushes raw ticks onto `price\_queue`. - `app/market/streaming.py` – consumes `price\_queue`, creates `BaseEvent`s. - `app/market/candles.py` – pure candle aggregation logic. - `app/market/signals.py` – simple threshold-crossing detector. - `app/risk/manager.py` – risk checks (size caps, daily loss, mode gating). - `app/services/event\_bus.py` – tiny in-process pub/sub bus. - `app/services/order\_workflow\_service.py` – validates order, persists, emits `OrderSubmitted`. - `app/services/reconciliation\_service.py` – syncs local DB with broker state (positions & orders). - `app/services/startup\_service.py` – orchestrates config, DB init, auth, reconciliation, websocket, then enables trading. - `app/orchestrator.py` – boots `StartupService` and runs `MarketStreamer` in background. - `app/api/routes.py` – FastAPI endpoints (`/health`, `/positions`). - `app/main.py` – entry point, starts FastAPI and the orchestrator. - `requirements.txt` – minimal pip requirements. - `.env.example` – template for required environment variables.

How the System Starts

1. **FastAPI starts** (`uvicorn app.main:app`). 2. `app.main` registers a `startup` event. 3. The startup event creates an `Orchestrator` instance and runs `orchestrator.run()` in a background task. 4. `Orchestrator.run()`: - Calls `StartupService.run()` → config → DB validation → `SessionManager.ensure\_valid()` → initial `ReconciliationService.run()` → opens the broker WebSocket. - After the WebSocket is connected, trading mode is set to `LIVE\_TRADING`. 5. `MarketStreamer.run()` continuously pulls ticks from `price\_queue`, turns each tick into a `BaseEvent`, and publishes it on the internal `EventBus`. 6. Consumers (e.g., a future order-executor) would subscribe to the `EventBus` and act on events such as `OrderSubmitted`, `SignalDetected`, etc.

Running the Project Locally

Prerequisites

- **Python 3.11+** (the repository was built with 3.14 on Arch Linux). - **Git** (to clone the repo if you haven't already). - **Docker & Docker Compose** (optional, for containerised deployment).

Step by Step

1. **Clone the repository** (if you aren't already in the folder):

```
bash
git clone <repo-url> capital_trader
cd capital_trader
```

2. **Create a virtual environment** (recommended to keep dependencies isolated):

```
bash
python -m venv .venv
source .venv/bin/activate
```

3. **Upgrade pip and install the requirements**:

```
bash
python -m pip install --upgrade pip
pip install -r requirements.txt
```

4. **Configure environment variables**:

```
bash
cp .env.example .env
# Edit .env with your values
nano .env
```

Required keys: - `CAPITAL\_API\_KEY` (or leave empty – the `SessionManager` will request a token via username/password). - `CAPITAL\_USERNAME` - `CAPITAL\_PASSWORD` - `DATABASE\_URL` – e.g., `postgresql+asyncpg://postgres:postgres@localhost/trading` - `ENVIRONMENT` – usually `demo` or `prod`. 5. **(Optional) Set up the PostgreSQL database** – you can use Docker:

```
bash
docker run -d --name pg -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -p 5432:5432 postgres:1
```

The first time the ORM accesses the DB it will automatically create the tables defined in `app/db/models`. 6. **Run the FastAPI server**:

```
bash
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
```

You should see logs similar to:

```
INFO: Started server process [...]
INFO: Uvicorn running on http://0.0.0.0:8000
```

7. **Verify the service** – open a browser or curl:

```
bash
curl http://localhost:8000/api/health
curl http://localhost:8000/api/positions
```

The health endpoint returns a JSON payload indicating the service is up. The positions endpoint returns the list of positions stored in the DB (empty on first run). 8. **Observe the background orchestration** – the console will show reconnection/backoff attempts if the placeholder broker URL (`https://api-capital.com/auth/login`) is not reachable. Replace the URLs in `app/broker/client.py` and `app/broker/websocket.py` with the real Capital.com endpoints, then restart the server.

Docker & Compose (quick start)

A minimal `Dockerfile` and `docker-compose.yml` are already present in the repo (you can find them under the project root). To build and run the whole stack:

```
bash
docker compose up --build
```

The compose file will start the app container and a PostgreSQL container. The app will wait for the DB to become reachable, then start exactly as described in the local steps.

Extending the Platform

- **Order Execution Consumer** – register a coroutine with `EventBus.subscribe()` that listens for `OrderSubmitted` events, calls `BrokerClient.request("POST", "/orders", ...)` , and updates the `Order` model based on the broker response. - **Metrics Exporter** – `app/metrics.py` already imports `prometheus\_client`. Expose `/metrics` via FastAPI and register counters (e.g., `order\_submitted\_total`). - **Alert Channels** – extend `app/alerts/notifications.py` with Slack or email senders, then have the `AlertService` publish to those channels. - **Testing** – add `pytest` and `pytest-asyncio` test modules under `app/tests/` that spin up an in-memory SQLite DB (`sqlite+aiosqlite:///memory:`) and verify repository behaviour, risk checks, and orchestrator startup. - **Reconciliation Frequency** – adjust the schedule in `StartupService` (currently called once at startup). For a recurring job you could use the `loop` skill or a background `asyncio` task that sleeps `5\*60` seconds.

License & Contributions

The code is provided **as-is** under the MIT license (feel free to replace with your preferred license). Contributions should follow the same architectural guidelines: keep layers thin, avoid heavy logic in the API layer, and always route DB work through repositories.

Generated by Claude Code on 2026-05-09.

Running the Capital.com Async Trading Platform

This document explains how to get the project up and running on your machine. It assumes you have **Python 3.11+** installed and a Unix-like shell.

1. Prerequisites

| Tool | Why we need it | |-----|-----| | `git` | To clone the repository (if you haven't copied the folder already). | | `python >=3.11` | The codebase uses modern async features and Pydantic 2. | | `virtualenv` (or `venv`) | To keep dependencies isolated from your system Python. | | `pip` | To install the packages listed in `requirements.txt`. | | **Optional** – Docker & Docker Compose | For containerised deployment (see the Docker section later). |

2. Clone the repository (skip if you already have the folder)

```
bash
git clone <repo-url> capital_trader
cd capital_trader
```

3. Create & activate a virtual environment

```
bash
python -m venv .venv
source .venv/bin/activate    # on Windows use .venv\Scripts\activate
```

You should now see the prompt prefixed with ``(.venv)``.

4. Install dependencies

First, upgrade ``pip`` (recommended):

```
bash
python -m pip install --upgrade pip
```

Then install everything:

```
bash
pip install -r requirements.txt
```

You will see a list of packages being downloaded – FastAPI, aiohttp, SQLAlchemy async, etc.

5. Configure environment variables

A template named ``env.example`` is provided. Copy it to ``env`` and fill in your real values:

```
bash
cp .env.example .env
nano .env # or any editor you prefer
```

The file must contain at least:

```
text
CAPITAL_API_KEY=          # optional - can be empty if you'll log in with username/password
CAPITAL_USERNAME=your_user
CAPITAL_PASSWORD=your_pass
ENVIRONMENT=demo          # or "prod"
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost/trading
LOG_LEVEL=INFO
```

If you don't have a PostgreSQL server yet, you can run one via Docker (see step 9).

6. (Optional) Spin up a PostgreSQL database with Docker

```
bash
docker run -d \
  --name pg \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  postgres:16-alpine
```

The ``DATABASE_URL`` in your ``env`` already points to this container (``localhost:5432``). The first time the app accesses the DB it will automatically create the tables (``positions``, ``orders``, ...) based on the SQLAlchemy models.

7. Start the FastAPI server (which also starts the background orchestrator)

From the project root, run:

```
bash
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Typical output:

```
INFO: Started server process [...]
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Application startup complete.
```

The orchestrator will now: 1. Load configuration & set up JSON logging. 2. Authenticate with Capital.com (or attempt to – you'll see backoff logs if the placeholder URL is still used). 3. Perform an initial reconciliation of local DB vs. broker state. 4. Open the market WebSocket and begin feeding ticks into `price\_queue`. 5. Switch the trading mode to `LIVE\_TRADING` once everything is ready.

8. Verify the service

Open another terminal (or keep the same one) and run a couple of curl commands:

```
bash
curl http://localhost:8000/api/health
curl http://localhost:8000/api/positions
```

You should receive JSON responses. `positions` will be an empty array on a fresh DB.

9. (Optional) Run everything via Docker Compose

A minimal `docker-compose.yml` is already present. To launch the full stack:

```
bash
docker compose up --build
```

The compose file defines two services: - `app` – builds the image from the repo, runs the same command as step 7. - `postgres` – the DB container used by `DATABASE\_URL`.

You can then hit `http://localhost:8000/api/health` from your host machine.

10. Next Development Steps

- **Replace placeholder URLs** in `app/broker/client.py` and `app/broker/websocket.py` with the real Capital.com endpoints. - **Implement an order execution consumer** that subscribes to `EventBus` for `OrderSubmitted` events, calls the broker API, and updates the `Order` model. - **Add metrics**: expose `/metrics` using `prometheus\_client` and register counters (e.g., order submissions, reconciliations). - **Write unit & integration tests** under `app/tests/` using `pytest-asyncio`. - **Extend the API** with POST endpoints for creating orders, managing alerts, etc.

11. Cleaning Up

When you are done, stop the server with `Ctrl+C`. If you started a Docker DB, bring it down with:

```
bash
docker compose down # or docker stop pg && docker rm pg
```

Delete the virtual environment if you no longer need it:

```
bash
deactivate
rm -rf .venv
```

Generated by Claude Code on 2026-05-09.